



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO THỰC HÀNH 02

Làm trơn ảnh và phát hiện biên cạnh

Nhóm: Myopia
MSHV 1: 25C11067 - Nguyễn Thái Thông
MSHV 2: 25C11035 - Trần Hạ Khánh Duy

TP. Hồ Chí Minh, 2026

1 Mục tiêu

Bài thực hành gồm hai nhóm nội dung chính: làm trơn ảnh và phát hiện biên cạnh. Phần làm trơn ảnh cài đặt và so sánh các bộ lọc mean filter, Gaussian filter và median filter. Phần phát hiện biên cạnh cài đặt và so sánh Sobel, Laplacian, Laplace of Gaussian (LoG) và Canny. Mỗi thuật toán được thực hiện bằng hai cách: cài đặt thủ công và dùng hàm OpenCV, sau đó đánh giá bằng hình minh chứng, MSE, thời gian chạy và bộ nhớ sử dụng.

2 Phân công đóng góp

MSHV	Họ tên	Công việc	Tỉ lệ
25C11067	Nguyễn Thái Thông	Cài đặt mean filter, Gaussian filter, Sobel; xây dựng hàm đo thời gian/bộ nhớ, kiểm thử kết quả thủ công so với OpenCV, thu thập hình minh chứng cho phần làm trơn và Sobel.	50%
25C11035	Trần Hạ Khánh Duy	Cài đặt median filter, Laplacian, LoG, Canny; tổng hợp số liệu MSE, nhận xét kết quả, lưu output, hoàn thiện notebook và viết báo cáo.	50%

3 Bảng tự đánh giá

Nội dung	Mức độ hoàn thành
Chuẩn bị môi trường, tải ảnh Lenna và hiển thị ảnh	100%
Làm trơn ảnh bằng mean filter	100%
Làm trơn ảnh bằng Gaussian filter	100%
Làm trơn ảnh bằng median filter	100%
Phát hiện biên cạnh bằng LoG	100%
Phát hiện biên cạnh bằng Sobel	100%
Phát hiện biên cạnh bằng Laplacian	100%
Phát hiện biên cạnh bằng Canny	100%
So sánh với hàm OpenCV	100%
Thu thập hình minh chứng và lưu kết quả	100%
Báo cáo	100%

4 Cơ sở lý thuyết tóm tắt

4.1 Làm trơn ảnh

Làm trơn ảnh là bước tiền xử lý dùng để giảm nhiễu hoặc giảm các chi tiết nhỏ trong ảnh. Bộ lọc trung bình thay mỗi pixel bằng giá trị trung bình của vùng lân cận nên làm mờ đều. Bộ lọc Gaussian dùng trọng số lớn hơn cho các điểm gần tâm kernel, vì vậy ảnh sau lọc thường tự nhiên hơn so với mean filter. Bộ lọc median thay pixel bằng trung vị của vùng lân cận, phù hợp khi ảnh có nhiều dạng điểm hoặc nhiễu muối tiêu.

4.2 Phát hiện biên cạnh

Biên cạnh là vùng có thay đổi cường độ sáng mạnh. Sobel dùng đạo hàm bậc một theo hai hướng x và y , sau đó tính độ lớn gradient. Laplacian dùng đạo hàm bậc hai nên nhạy với các vùng biến thiên mạnh nhưng cũng dễ bị ảnh hưởng bởi nhiễu. Canny là phương pháp nhiều bước gồm làm trơn Gaussian, tính gradient, non-maximum suppression, ngưỡng kép và nối biên bằng hysteresis.

5 Đánh giá kết quả

5.1 Ảnh đầu vào

Ảnh Lenna được tải và resize về kích thước nhỏ hơn để các thuật toán thủ công chạy nhanh. Ảnh màu được chuyển sang ảnh xám trước khi áp dụng các thuật toán phát hiện biên.

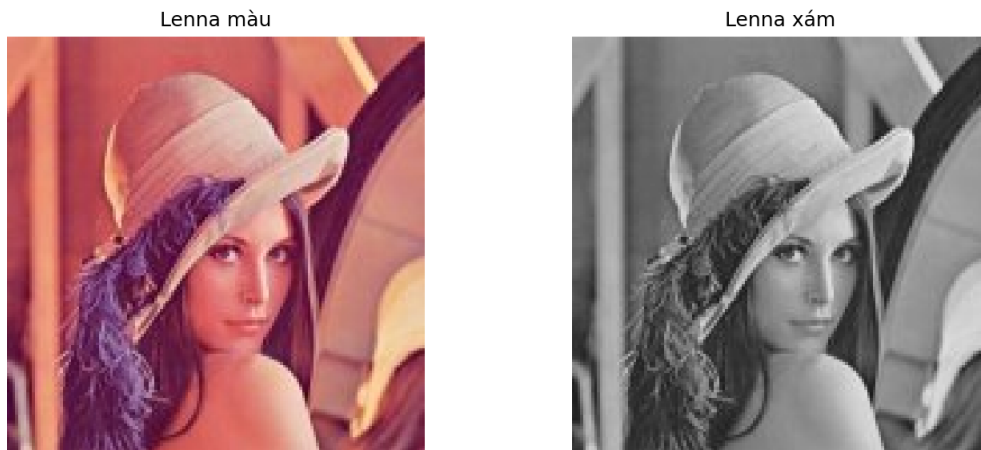


Figure 1: Ảnh Lenna màu và ảnh Lenna xám.

5.2 Làm trơn ảnh

Mean filter làm giảm nhiễu bằng cách lấy trung bình các pixel trong vùng lân cận. Trong notebook, nhóm hiển thị thêm ảnh sai khác giữa kết quả thủ công và OpenCV; ảnh này gần như tối hoàn toàn, cho thấy hai kết quả rất sát nhau. MSE đo được là 0.0000, thời gian chạy của bản thủ công là 0.365850s với bộ nhớ đỉnh 1080.70KB, trong khi OpenCV chỉ mất 0.001042s và 66.35KB. Điều

này cho thấy mean filter thủ công cho kết quả gần như trùng khớp nhưng chi phí tính toán cao hơn nhiều do phải duyệt từng cửa sổ bằng vòng lặp Python.

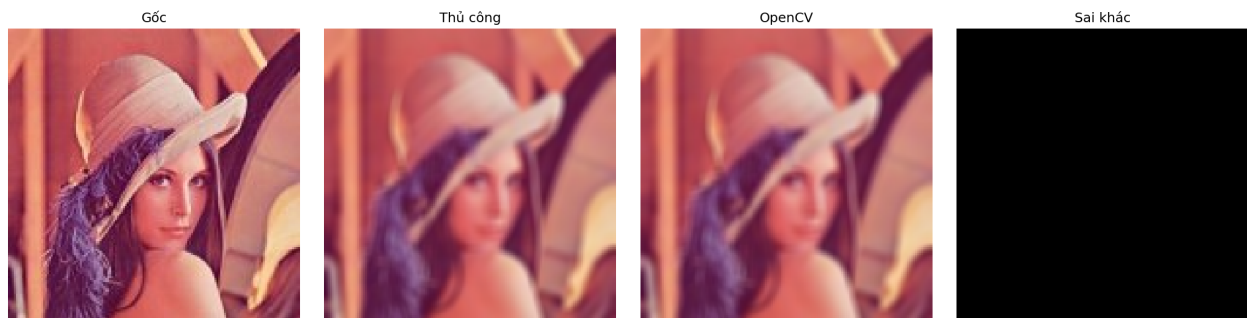


Figure 2: So sánh mean filter thủ công và OpenCV.

Gaussian filter dùng kernel có trọng số theo phân phối Gaussian. So với mean filter, bộ lọc này thường giữ cấu trúc ảnh mềm và tự nhiên hơn vì pixel gần tâm có ảnh hưởng lớn hơn pixel xa tâm. Kết quả thực nghiệm cho MSE 0.0053, nghĩa là ảnh thủ công và OpenCV vẫn rất gần nhau nhưng đã có sai khác lớn hơn mean filter do bước tạo kernel, tích chập và làm tròn. Bản thủ công chạy trong 0.222884s với bộ nhớ đỉnh 802.26KB, còn OpenCV mất 0.001628s và 66.35KB. Ảnh sai khác cho thấy khác biệt chủ yếu xuất hiện nhẹ ở các vùng chuyển mức xám và biên mềm.

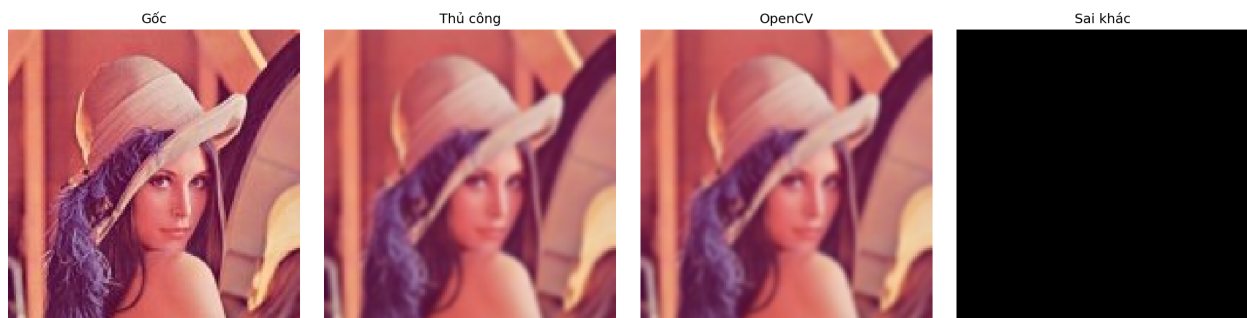


Figure 3: So sánh Gaussian filter thủ công và OpenCV.

Median filter lấy trung vị trong vùng lân cận, do đó có khả năng loại bỏ nhiễu điểm tốt. Kết quả có xu hướng giữ biên tốt hơn mean filter trong một số vùng ảnh. Trong ba phép làm trơn, median filter có sai khác lớn nhất giữa hai cách cài đặt với MSE 0.4045. Nguyên nhân hợp lý là thao tác lấy trung vị phụ thuộc mạnh vào cách xử lý biên và cách sắp xếp giá trị trong từng cửa sổ. Bản thủ công cần 0.817596s và 216.54KB, còn OpenCV chỉ mất 0.000487s và 66.18KB, nên đây cũng là phép lọc cho thấy rõ nhất lợi thế tối ưu của thư viện.

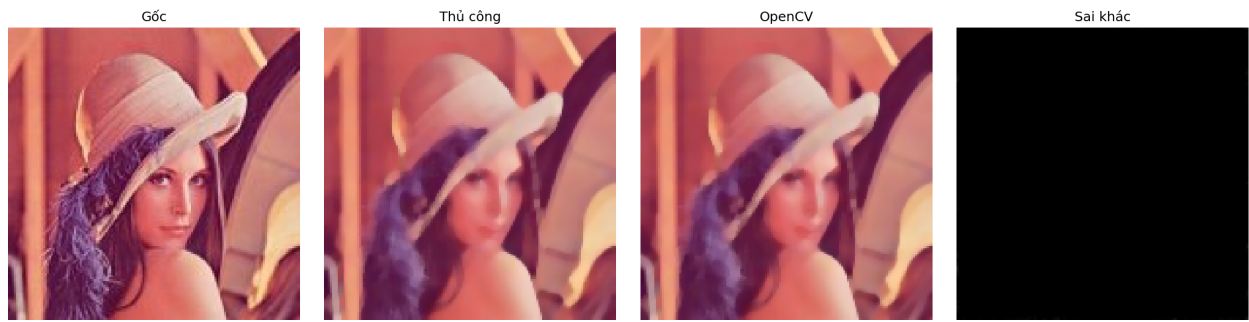


Figure 4: So sánh median filter thủ công và OpenCV.

5.3 Phát hiện biên cạnh

Sobel phát hiện biên bằng cách tính gradient theo hai hướng. Ảnh biên thể hiện rõ các vùng thay đổi cường độ sáng như đường viền khuôn mặt, nón và vai áo trong ảnh Lenna.

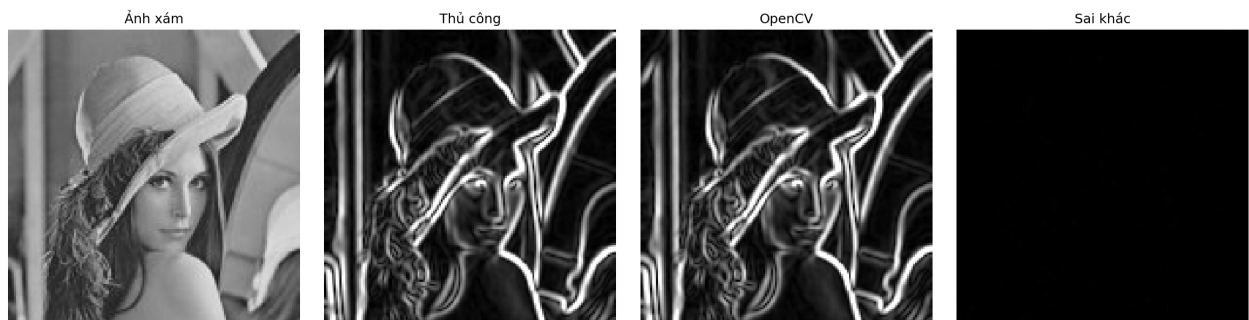


Figure 5: So sánh phát hiện biên Sobel thủ công và OpenCV.

Laplacian dùng đạo hàm bậc hai nên phản ứng mạnh tại vùng có thay đổi cường độ đột ngột. Trước khi áp dụng Laplacian, ảnh được làm trơn Gaussian để giảm bớt nhiễu.

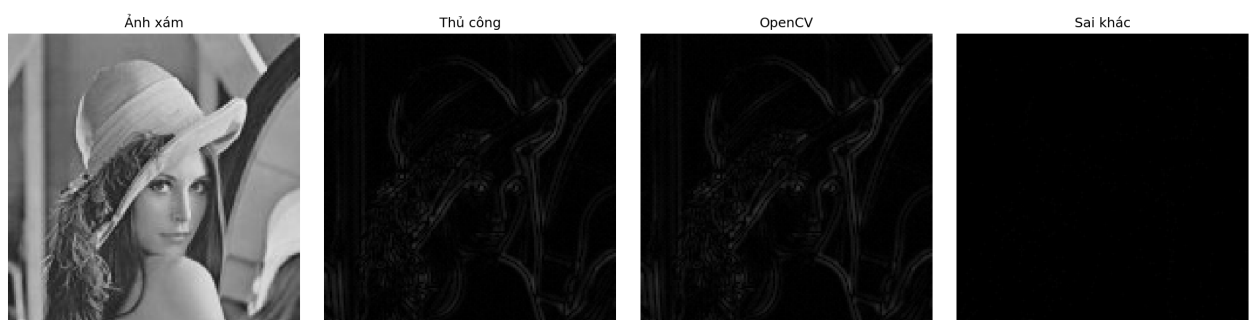


Figure 6: So sánh phát hiện biên Laplacian thủ công và OpenCV.

LoG (Laplace of Gaussian) kết hợp bước làm trơn Gaussian với toán tử Laplacian. Vì vậy, phương pháp này vừa thuộc nhóm phát hiện biên, vừa thể hiện rõ vai trò của bước làm trơn trong việc hạn chế nhiễu trước khi lấy đạo hàm bậc hai. Kết quả đo được cho MSE 72.2126, cao hơn đáng kể so với Laplacian thường, cho thấy cách cài đặt thủ công và OpenCV khác nhau rõ hơn khi dùng kernel 8-neighborhood và tham số Laplacian của thư viện. Thời gian chạy của bản thủ công là 0.342847s với 293.61KB, còn OpenCV mất 0.001922s và 88.54KB.

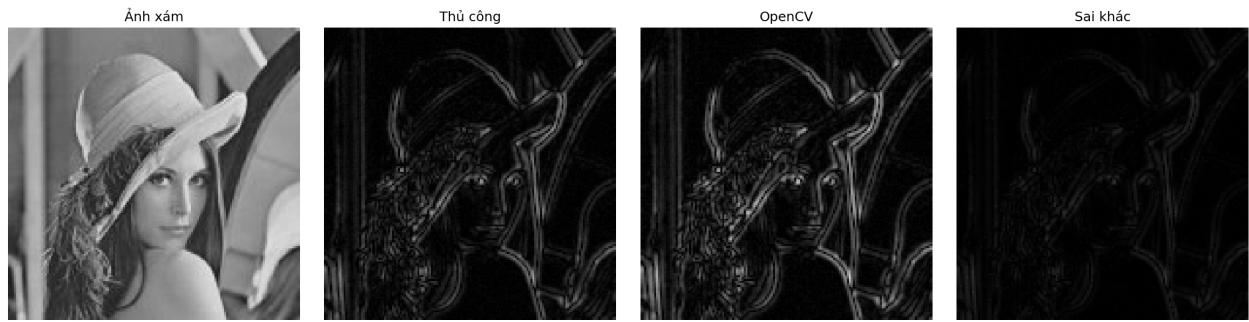


Figure 7: So sánh phát hiện biên LoG thủ công và OpenCV.

Canny tạo biên mảnh hơn nhờ bước non-maximum suppression và ngưỡng kép. Kết quả thủ công có thể khác OpenCV nhiều hơn Sobel và Laplacian vì OpenCV tối ưu nhiều chi tiết nội bộ trong quá trình làm mảnh và nối biên.

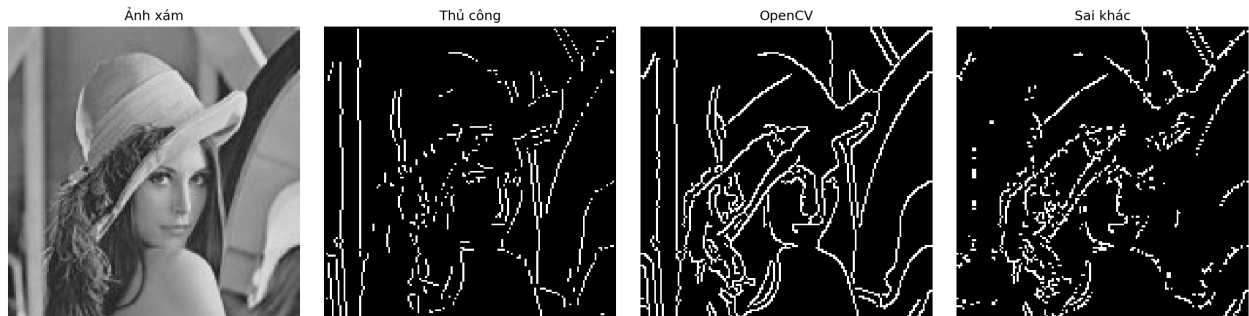


Figure 8: So sánh phát hiện biên Canny thủ công và OpenCV.

5.4 So sánh kết quả

Bảng dưới đây được cập nhật theo kết quả lưu từ lần chạy notebook gần nhất. Nhóm làm trơn gồm ba bộ lọc (mean, Gaussian, median), còn LoG được tính vào nhóm phát hiện biên.

Phép xử lý	MSE	Thủ công	OpenCV	Nhận xét
Mean filter	0.0000	0.365850s 1080.70KB	0.001042s 66.35KB	Hai kết quả gần như trùng nhau; ảnh sai khác rất nhỏ và chủ yếu đến từ làm tròn hoặc xử lý biên.
Gaussian filter	0.0053	0.222884s 802.26KB	0.001628s 66.35KB	Kết quả trực quan vẫn rất gần nhau; sai khác tăng nhẹ do kernel Gaussian và phép tích chập số thực.
Median filter	0.4045	0.817596s 216.54KB	0.000487s 66.18KB	Median thủ công chậm nhất trong ba phép làm trơn vì phải tính trung vị cho từng cửa sổ ảnh.

LoG	72.2126	0.342847s 293.61KB	0.001922s 88.54KB	LoG kết hợp làm trơn Gaussian và Laplacian nên nhạy với khác biệt trong kernel và cách hiện thực của OpenCV.
Sobel	0.2713	0.511674s 470.94KB	0.001183s 397.21KB	Biên chính được phát hiện rõ; sai khác nhỏ có thể do padding và kiểu dữ liệu.
Laplacian	0.3692	0.338727s 293.96KB	0.012127s 88.54KB	Phản ứng mạnh tại vùng biến thiên cường độ; nhạy với nhiễu nên cần Gaussian trước.
Canny	4629.7798	0.560139s 563.33KB	0.003300s 44.40KB	Sai khác có thể lớn hơn vì Canny gồm nhiều bước và OpenCV có tối ưu nội bộ.

Nhìn chung, cài đặt thủ công giúp hiểu rõ nguyên lý của từng thuật toán, nhưng tốc độ chậm hơn do sử dụng vòng lặp Python. OpenCV phù hợp hơn khi cần xử lý nhanh, ổn định và triển khai thực tế. Trong các kết quả đo được, Mean filter có MSE thấp nhất, còn LoG và Canny cho sai khác lớn hơn do quy trình xử lý phức tạp hơn và phụ thuộc nhiều vào chi tiết hiện thực.

6 Kết luận

Bài thực hành hoàn thành hai nhóm nội dung: làm trơn ảnh và phát hiện biên cạnh. Ở phần làm trơn, notebook khảo sát ba bộ lọc gồm mean filter, Gaussian filter và median filter. Các bộ lọc làm trơn giúp giảm nhiễu và chuẩn bị ảnh tốt hơn cho bước phát hiện biên. Các thuật toán phát hiện biên cho thấy nhiều cách tiếp cận khác nhau: Sobel dựa trên gradient bậc một, Laplacian dựa trên đạo hàm bậc hai, LoG kết hợp làm trơn với Laplacian để phát hiện biên, còn Canny gồm nhiều bước để tạo biên mảnh và rõ hơn.

Kết quả so sánh cho thấy OpenCV nhanh hơn đáng kể so với cài đặt thủ công. Tuy nhiên, phần cài đặt thủ công vẫn cần thiết trong bài thực hành vì giúp kiểm chứng công thức, hiểu rõ xử lý kernel, padding, gradient, ngưỡng và quá trình tạo ảnh biên.